Algoritmos Heurísticos: El Algoritmo del Gran Diluvio

Fundamentos, Formulación Estándar, Multi-start y Enfoque Poblacional

Optimización Heurística

27 de octubre de 2026

Contenido

1. Introducción e Intuición
2. Formulación Matemática
3. Implementaciones en Python
4. Estrategias Multimodales
5. Conclusiones

Origen del Algoritmo del Gran Diluvio

Origen Histórico

Propuesto por el matemático alemán **Gunter Dueck** en 1993 en su trabajo seminal *"New Optimization Heuristics: The Great Deluge Algorithm"*.

Analogía Topográfica

- Imagina que la función de costo representa la altitud de un terreno.
- Buscamos el punto más bajo (mínimo global en minimización).
- Un nivel de agua L (*level*) sube o brolla constantemente.
- El algoritmo realiza pasos de exploración siempre y cuando la nueva posición esté **por encima del nivel de agua** (sin "mojarse los pies").

Simulated Annealing vs. Gran Diluvio

2gray!10white

Característica	**Simulated Annealing**	**Gran Diluvio (Great Deluge)**
Aceptación	Probabilística ($r < e^{-\Delta E/T}$)	Determinista ($f(x) \leq L$)
Parámetros	Temp. inicial T_0, tasa α	Nivel inicial L_0, tasa de descenso UP
Evaluación	Evalúa exponentes e^x y aleatorios	Comparación lógica directa ($<$)
Comportamiento	Acepta empeoramientos con prob. decreciente	Acepta cualquier punto dentro del límite actual

Criterio de Aceptación Determinista

Para un problema de minimización de una función $f(x)$ con $x \in \mathbb{R}^n$:

$$x_{\text{nuevo}} = x_{\text{actual}} + \Delta x, \quad \Delta x \sim \mathcal{U}(-\epsilon, \epsilon) \tag{1}$$

El punto x_{nuevo} es aceptado si cumple la condición del umbral:

$$f(x_{\text{nuevo}}) \leq L \tag{2}$$

Descenso del Nivel de Agua (L)

En cada iteración k, el límite L se reduce de forma constante:

$$L_{k+1} = L_k - UP \tag{3}$$

donde UP (Univariate Parameter / Rain rate) se calcula como:

$$UP = \frac{L_{\text{inicial}} - L_{\text{objetivo}}}{N_{\text{iteraciones}}} \tag{4}$$

Implementación Estándar (Vectorial 2D)

```python
import numpy as np

def gran_diluvio_2d(f_objetivo, x_inicial, max_iter=10000):
    x_actual = np.array(x_inicial, dtype=float)
    costo_actual = f_objetivo(x_actual)

    # 1. Configurar nivel de agua inicial L
    L = costo_actual + 20.0
    UP = (L - 0.0) / max_iter

    x_mejor, costo_mejor = np.copy(x_actual), costo_actual

    for i in range(max_iter):
        # Generar vecino aleatorio en 2D
        x_nuevo = x_actual + np.random.uniform(-0.3, 0.3, size=2)
        costo_nuevo = f_objetivo(x_nuevo)

        # Criterio del Gran Diluvio
        if costo_nuevo <= L:
            x_actual, costo_actual = x_nuevo, costo_nuevo
            if costo_nuevo < costo_mejor:
                x_mejor, costo_mejor = np.copy(x_nuevo), costo_nuevo

        L -= UP # Descenso del nivel de agua

    return x_mejor, costo_mejor
```

Optimizadores Multimodales

Limitación del Algoritmo Estándar

La versión básica es una metaheurística de **trayectoria única**. Solo puede converger a un único mínimo (local o global) por corrida.

Para identificar múltiples mínimos en paisajes multimodales (ej. Función de Himmelblau), se emplean dos estrategias principales:

1. **Estrategia Multi-start (Lanzamientos Múltiples):** Ejecuciones secuenciales e independientes desde diferentes puntos iniciales.
2. **Estrategia Multi-agente (Poblacional / Paralela):** Múltiples individuos explorando en paralelo bajo el mismo nivel de agua L.

Estrategia 1: Multi-start

Concepto

Se repite la corrida del algoritmo M veces, muestreando x_0 de manera uniforme dentro de los límites de búsqueda.

Algoritmo Multi-start

- Se mantiene un registro de todas las soluciones encontradas.
- Se aplican filtros de proximidad (distancia euclidiana) para eliminar duplicados y catalogar los mínimos locales/globales verdaderos.

$$X_{\text{iniciales}} = \{x_0^{(1)}, x_0^{(2)}, \ldots, x_0^{(M)}\} \quad \Longrightarrow \quad \text{Ejecutar Gran Diluvio por cada } x_0^{(i)} \tag{5}$$

Estrategia 2: Algoritmo Poblacional / Multi-agente

```python
def gran_diluvio_poblacional(f_objetivo, num_agentes=30, max_iter=8000, bounds=(-5, 5)):
    # Poblacion inicial en 2D y evaluacion de costos
    poblacion = np.random.uniform(bounds[0], bounds[1], size=(num_agentes, 2))
    costos = f_objetivo(poblacion)

    L = np.max(costos) + 10.0
    UP = (L - 0.0) / max_iter

    for i in range(max_iter):
        # Propuesta paralela de pasos para toda la poblacion
        pasos = np.random.uniform(-0.3, 0.3, size=(num_agentes, 2))
        propuestas = np.clip(poblacion + pasos, bounds[0], bounds[1])
        costos_propuestos = f_objetivo(propuestas)

        # Criterio del Diluvio Vectorial
        aceptados = costos_propuestos <= L

        # Actualizacion masiva de agentes aprobados
        poblacion[aceptados] = propuestas[aceptados]
        costos[aceptados] = costos_propuestos[aceptados]

        L -= UP # Todos los agentes comparten el nivel L

    return poblacion, costos
```

Comparativa de Estrategias Multimodales

2gray!10white

Métrica	**Multi-start**	**Multi-agente (Poblacional)**
Naturaleza	Repeticiones secuenciales independientes	Evolución colectiva paralela
Memoria	$O(1)$ por corrida	$O(N \cdot D)$ donde N es la población
Coordinación	Ninguna	Comparten el umbral de agua L
Ventaja	Extremadamente fácil de paralelizar en GPU/CPU	Segmentación natural en múltiples valles

Conclusiones

- El **Gran Diluvio** es una alternativa determinista eficiente a Simulated Annealing que prescinde de la evaluación de funciones exponenciales.
- Su único parámetro crítico de calibración es la tasa de descenso UP.
- Para funciones con un solo valle es rápido y efectivo.
- En paisajes multimodales, el marco **Multi-start** o el paradigma **Multi-agente/Poblacional** permiten descubrir todos los mínimos locales y globales de manera robusta.